

Trellis: a graph language and engine built for agents

Design briefing and decision document

LossMetricKG project

August 24, 2026

Abstract

This document collects everything needed to make the remaining design decisions for an agent-optimized graph database language and its engine. It is organised as a series of chapters, one per design area. Each chapter tells the story in plain terms, lays out the evidence, and ends in explicit decision boxes: the options, their trade-offs, and a recommendation. Decisions already made in earlier discussion are marked as settled and restated so the whole picture sits in one place. A consolidated decision checklist and an annotated reading list close the document. The working name used throughout is **Trellis**: a structure that supports something as it grows, which is exactly what this language does for an agent-built graph.

Contents

1	Why this project exists	3
1.1	What is already settled	4
2	The language surface	4
2.1	The five failure modes, and what kills each one	4
2.2	The evidence on LLM-friendly syntax	5
2.3	A sketch of what a Trellis read looks like	5
2.4	Decisions to make	6
3	Node classes and granularity	7
3.1	The story	7
3.2	Decisions to make	8
4	Writes, deduplication, and schema evolution	9
4.1	The write surface	9
4.2	Deduplication	9
4.3	Schema evolution	9
4.4	Decisions to make	10
5	Failure nodes and the maintenance loop	10
5.1	The story	10

5.2	Decisions to make	11
6	Result serialization and token budgets	12
6.1	The story	12
6.2	Decisions to make	12
7	Engine architecture	13
7.1	The story, in plain terms	13
7.2	Partitioning: the graph-specific hard part	14
7.3	Decisions to make	14
8	Evaluation plan	15
9	Consolidated decision checklist	15
10	Annotated reading list	16

1 Why this project exists

When an AI agent wants to ask a graph database a question today, it has to write a query in a language made for humans, usually Cypher, the query language of Neo4j. Even the best frontier models get roughly forty percent of these queries wrong on realistic schemas: Claude 3.5 Sonnet reaches 61.6% execution accuracy on CypherBench, GPT-4o 60.2%, and no model under ten billion parameters clears 20% [arXiv 2412.18702]. Neo4j’s own Text2Cypher benchmark puts the best models near 30% on execution-based exact match. SPARQL zero-shot sits under 4% for most models, and accuracy across query languages tracks Stack Overflow post counts rather than any measure of language quality [arXiv 2411.05521].

The industry’s reaction has been to give up on query languages entirely. Every production agent-memory system (Zep/Graphiti, Mem0, Cogniee, Anthropic’s MCP memory server) hides the graph behind a handful of search tools; the agent never sees the graph and never writes a query [arXiv 2501.13956, 2504.19413]. Memgraph explicitly tells customers to wrap curated queries in named tools instead of letting the model generate Cypher. That approach works, but it throws away expressivity: ad hoc aggregation, multi-hop composition, and every new question shape needs a new hand-written tool.

Nobody has tried the third option: fix the language. Three findings make that credible. First, the failure modes are structural, not random; a language can make each one impossible by design. Second, the Anka result shows a purpose-built DSL with zero pretraining exposure beating Python by forty percentage points on multi-step tasks, purely by removing error-prone degrees of freedom [arXiv 2512.23214]. Third, constrained decoding now makes syntax validity essentially free, so design effort can go entirely into semantics and token economics.

Trellis goes further than a query language. This project also owns the write side and the engine, because its target workload is not humans querying a finished graph. It is agents *building and maintaining* a graph at scale, which is where the LossMetricKG construction work plugs in directly.

Plain words

Cypher: Neo4j’s graph query language, the most common one. **Schema:** the list of allowed node types, edge types, and properties in a graph. **DSL:** domain-specific language, a small language built for one job. **Constrained decoding:** forcing a model, token by token, to only emit output that matches a grammar, so invalid syntax cannot appear at all. **Execution accuracy:** fraction of generated queries that return the correct answer when run, the strictest common metric.

1.1 What is already settled

Already settled

From prior discussion, the following are fixed and this document does not reopen them:

1. **Engine in scope.** Trellis is not only a compiler over existing databases; a scalable engine is part of the project, with the architecture designed for billions of datapoints.
2. **General-purpose workload.** Three workloads carry equal weight: agent KG construction, agent memory, and analytics over existing large graphs.
3. **Agent-explicit granularity.** The language ships a minimal set of primitives; agents decide when nodes split or merge. The system executes, keeps lineage, and enforces invariants; it may surface signals but never acts alone.
4. **Dedup: system detects, agent resolves.** The engine continuously finds duplicate candidates; agents resolve them with explicit verbs.
5. **Failure nodes defined broadly.** Both the five LLM query-generation failure modes (attacked by language design) and four runtime node-health signals: construction-loss, query-failure attribution, structural anomalies, staleness.
6. **Full structural writes.** Agents write nodes and edges directly through DSL verbs, with system-side guards. Not the episodic-ingest model of existing memory systems.
7. **Schema evolves, gated.** Humans seed base classes; agents propose subclasses through a language verb; the system gates promotion.

2 The language surface

2.1 The five failure modes, and what kills each one

LLMs fail at Cypher in predictable, well-documented ways. Each failure mode has a known language-level countermeasure, and Trellis should be judged by whether it makes each mistake *impossible*, not merely less likely.

Failure mode	Language-level fix
Reversed relationship direction. The most-cited Cypher failure; whole repair systems exist just for this.	Remove arrow glyphs entirely. Edges are written with named roles (author , work) so there is no direction to reverse.
Schema hallucination: invented labels and properties.	Schema-closed vocabulary. The grammar handed to constrained decoding is generated <i>from the live schema</i> , so a nonexistent label cannot be emitted.
Pattern is valid but means something other than the question.	Canonical forms: exactly one way to write each operation, plus named intermediates so every step is inspectable.
Aggregation logic errors (Cypher’s implicit grouping).	Explicit, staged aggregation: group, then aggregate, as separate named steps.
Long-range bracket matching and deep nesting.	Line-oriented, keyword-delimited blocks. No braces to balance across distance.

Sources: CypherBench error taxonomy [arXiv 2412.18702], direction-repair literature [arXiv

2606.04645, 2409.04181].

2.2 The evidence on LLM-friendly syntax

Convergent heuristics across Anka, SPEAC, BAML, TOON, grammar prompting, and schema-adaptation studies:

- **Verbose keywords over symbols.** English words leverage the language prior; symbol-dense operators concentrate errors.
- **Canonical forms.** One way to say each thing removes decision points and makes output checkable.
- **Named intermediates over chaining.** Variable shadowing and chaining confusion caused 69% of Python’s failures in the Anka study. Every step binds an explicit name.
- **Locality.** Syntax errors concentrate in long-range delimiter matching; prefer line-oriented, keyword-terminated blocks.
- **Redundancy as self-check.** Declare counts and field headers up front in the TOON `items[3]` style; both model and validator get a checkable invariant.
- **Reasoning space before answers** in any fixed output schema; field order measurably matters.
- **Teach by example, not grammar prose.** Models learn DSLs from worked examples far better than from grammar descriptions. Documentation is an example corpus.
- **Mind the prompt tax.** A novel syntax must save more tokens than its in-context tutorial costs [arXiv 2603.03306].
- **Token-boundary alignment.** Delimiters on natural token boundaries; whole words, newline-terminated.

Two proven routes past the zero-corpus cold start: grammar prompting, a compact BNF grammar plus exemplars in context [arXiv 2305.19234], and familiar-surface compilation, where the model writes a subset of a language it knows and a compiler repairs it into the target, which raised parse success from 3% to 84.8% in SPEAC [arXiv 2406.03636]. Note also the Anka caveat: a designed DSL only wins on tasks with five or more operations. On one-to-two hop lookups a new language shows no advantage. Trellis’s pitch must centre on multi-hop, compositional, and construction workloads.

Plain words

BNF / grammar: a formal, machine-readable description of exactly which strings are valid programs. **Prompt tax:** the context tokens spent teaching the model your language each session, which must be earned back in saved output tokens or accuracy. **Named intermediate:** giving each step’s result a variable name instead of piping results invisibly into the next step.

2.3 A sketch of what a Trellis read looks like

Illustrative only, not final syntax. The question: “which suppliers provide parts used in products launched after 2024, and how many parts each?”

```

find products where launch_year > 2024 as recent
follow recent uses part as parts
follow parts supplied_by supplier as sups
group sups by supplier as per_supplier
count per_supplier.parts as n_parts
return supplier.name, n_parts limit 20 budget 2000 tokens

```

Every line is one step, every step binds a name, edges are traversed by role name with no arrows, aggregation is staged, and the result carries an explicit token budget. This shape is what the decisions below refine.

2.4 Decisions to make

Decision #1

Verb count and surface size. The research points to 8–12 verbs for reads. With writes, granularity, dedup, and schema evolution added, the full surface will be nearer 15–20 verbs.

- A. *Minimal (about 12 verbs total).* One page teaches everything; smallest prompt tax; some operations get awkward.
- B. *Two-tier (about 12 core + about 8 maintenance verbs, taught separately).* Query agents load only the core tutorial; construction agents load both. Keeps each prompt small.

Recommended.

- C. *Full flat surface (about 20 verbs, one tutorial).* Simplest spec, biggest per-session tax.

Decision #2

Edge direction encoding.

- A. *Role-named only.* Every edge type declares role names for both ends (`authored_by(work, author)`); queries mention roles, direction cannot be stated and so cannot be wrong. Strongest fix; requires every schema to name roles well. **Recommended.**
- B. *Direction-explicit keywords.* `from X to Y` instead of arrows. Direction still exists and can still be reversed, but the mistake is more visible.
- C. *Both allowed.* Violates canonical forms; not recommended.

Decision #3

Cold-start strategy for generation quality. Not mutually exclusive; choose the launch combination.

- A. *Grammar prompting.* Compact grammar + worked examples in context. Cheapest, works everywhere. **Recommended as baseline.**
- B. *Schema-conditioned constrained decoding.* Generate a grammar from the live schema so invalid labels are unemittable. Needs decode-side control (available in open-weight serving and increasingly in APIs). **Recommended wherever available.**
- C. *Familiar-surface compilation.* Accept a Cypher-like or SQL-like subset and repair into Trellis. Good adoption ramp, doubles the compiler surface; defer to v2.
- D. *Fine-tune / skill file.* An 8B fine-tune matched zero-shot frontier models on GQL; a distributable “skill” document is the cheap version. Do after the eval harness exists.

Decision #4

Pattern-matching semantics. This is a real design axis that changes both result counts and complexity class: homomorphism vs trail vs vertex-distinct semantics; trail semantics can go NP-hard. GQL lets users pick per query (WALK/TRAIL/ACYCLIC).

- A. *Fix one default (homomorphism, the cheap one) and expose nothing.* Simplest, canonical, predictable cost. **Recommended for v1.**
- B. *Per-query keyword like GQL.* More expressive, another decision point for the model, invites the exact “valid but wrong” failures we are eliminating.

3 Node classes and granularity

3.1 The story

Most graph models force one grain: a node is an entity, full stop. Real agent workloads do not arrive at one grain. An agent ingesting a report may first store an entire table as a single node (cheap, fast, lossless), then later, when queries start hitting that table, break it into one node per row, or per cell-level fact. Trellis makes grain a first-class, agent-controlled property of a node rather than a modelling accident.

The mechanism has three parts. First, a **class system**: every node belongs to a class; classes form a tree from human-seeded base classes down through agent-derived subclasses. A class declares its properties, its allowed edge roles, and, critically, its **allowed granularity states**. Second, **granularity states**: a node is in exactly one state, for example **blob** (an opaque payload such as a whole table), **composite** (has structured children), or **atom** (a single fact, no children). Third, **refinement verbs**: **refine** splits a node into children under a declared mapping; **compact** replaces a set of nodes with a summary node. Both keep full lineage: the parent persists as a lineage record, every child knows its origin, and any query answer can be traced back to the blob it came from.

The agent holds the verbs, per the settled agent-explicit principle. The system’s role is to enforce class-declared invariants (a class can forbid **blob** state, or cap children), to keep lineage automatically, and to surface signals (“this blob node is traversed 400 times/day; refining it would make those queries indexable”).

Plain words

Grain / granularity: how much stuff one node holds; a whole table is coarse grain, one fact is fine grain. **Lineage:** the recorded history of where data came from, so a fine-grained node can point back to the blob it was extracted from. **Invariant:** a rule the system promises never to let become false.

3.2 Decisions to make

Decision #5

Granularity state model.

- A. *Three fixed states: blob, composite, atom.* Simple, canonical, easy to teach in the tutorial. **Recommended.**
- B. *Class-defined arbitrary states.* Each class declares its own state machine. Maximum flexibility, but agents must learn per-class states, which reintroduces schema-shaped hallucination risk.
- C. *No states, just parent-child.* Grain becomes emergent structure. Cheapest, but the system cannot enforce grain invariants or generate grain-aware grammars.

Decision #6

Refinement mapping: how does refine know how to split a blob?

- A. *Agent supplies the mapping inline* (target class + extraction spec per child). Pure agent-explicit; mapping quality varies with the agent. **Recommended, with mappings saved and reusable as named artifacts.**
- B. *Class-declared refiners.* Subclass ships a canonical refiner; agents just invoke it. Consistent, but someone must author refiners ahead of need.
- C. *System-inferred (LLM-powered) refinement.* System runs its own extraction. Convenient, but moves construction loss out of the agent's control and blurs accountability.

Decision #7

Lineage retention at billions of nodes. Lineage is essential for trust and undo, and it is unbounded growth.

- A. *Full lineage forever, in cold storage.* Parents and mappings are moved to cheap object storage, always reachable, never in the hot path. **Recommended given the disaggregated engine (Section 6).**
- B. *Configurable retention windows per class.* Saves cold storage, breaks the “every answer traceable” guarantee.
- C. *Lineage only for refined/merged nodes, not plain writes.* Half measure; plain writes are exactly where provenance questions come from.

4 Writes, deduplication, and schema evolution

4.1 The write surface

Settled: agents write structurally. The candidate verb set: **assert** (create or update a node/edge with stated provenance), **merge** (resolve two nodes into one, with property reconciliation), **refine** / **compact** (granularity, above), **retire** (temporal invalidation: the node stops being current but remains queryable historically), and **derive class** (schema evolution, below). Every write verb returns a structured receipt: what changed, what guards fired, and, crucially, any duplicate candidates the write triggered. The receipt is the system’s half of the conversation; it is how “system detects, agent resolves” actually reaches the agent.

This write model is the novel part relative to every production memory system, all of which converged on episodic ingestion because they did not trust the model to write structurally. The bet, supported by the Anka result, is that a write language designed to be un-fumble-able changes that calculus. The guards make the bet safer: schema validation at parse time (unemittable if constrained decoding is on), class invariants at execution, dedup candidates surfaced on every assert.

4.2 Deduplication

Settled: the system detects continuously, the agent resolves explicitly. At billions of nodes, detection is a pipeline, not a lookup:

1. **Blocking:** cheap keys (class + normalised name prefix + salient property hashes) partition candidates so the quadratic comparison space collapses. Runs at write time.
2. **Embedding similarity:** an approximate-nearest-neighbour index per class over node embeddings catches non-obvious duplicates. Refreshed asynchronously.
3. **Candidate ledger:** pairs above threshold enter a per-class queue with evidence attached (matching properties, shared neighbourhoods, embedding score). Exposed to agents through the read language like any other data: `find dup_candidates where class = supplier order by score`.
4. **Resolution:** agent issues **merge** (with a property-reconciliation policy) or **distinct** (a durable “these are different” assertion that suppresses the pair and, ideally, teaches the blocker).

Merges must be undoable, which lineage gives us: one node survives (its id stays stable, the absorbed id becomes an alias so references never dangle), and lineage records the full pre-merge state of both parents, so any merge can be unwound.

4.3 Schema evolution

Settled: gated agent evolution. The flow: an agent hits an entity that fits no class well and issues `derive class supplier_broker from supplier with {...}`. The new subclass enters **provisional** status: instances can be created, but the class is marked, quota-limited, and not yet part of the stable grammar. The system gates **promotion:** dedup against existing classes (is this actually **distributor?**), minimum stable-instance count, no pending contradictions. On promotion the schema-closed grammar regenerates and every agent’s constrained decoder picks up the new vocabulary. Demotion (class merged away) follows the same lineage rules as node merges.

Plain words

Blocking: a classic dedup trick; instead of comparing everything with everything, first bucket records by a cheap key and only compare within buckets. **ANN index:** approximate nearest neighbour; a data structure that finds “things with similar embeddings” fast without checking every item. **Provenance:** who said this and where it came from. **Quota:** a hard cap the system enforces on provisional things so a confused agent cannot flood the schema.

4.4 Decisions to make

Decision #8

Merge property reconciliation. When two nodes merge and properties conflict:

- A. *Agent states policy per merge* (**prefer newest** / **prefer source X** / explicit values). Canonical-form friendly; one more thing to generate. **Recommended, with prefer newest the default.**
- B. *Keep both, versioned.* Nothing lost, queries must then handle multi-valued properties everywhere.
- C. *System picks by provenance trust scores.* Requires a trust model first; defer.

Decision #9

Dedup detection latency guarantee.

- A. *Synchronous blocking-stage check on every assert; embedding stage async.* Write receipt always carries exact-key candidates immediately; fuzzy candidates arrive in the ledger later. Bounded write latency, small dupe window for fuzzy cases. **Recommended.**
- B. *Fully async.* Fastest writes, largest dupe window; agents may build on nodes that later merge.
- C. *Fully synchronous including ANN.* Strongest guarantee; ANN lookup on every write at billions scale is a real cost and a tail-latency risk.

Decision #10

distinct assertions: scope and learning.

- A. *Pair-level suppression only.* Simple, correct, no generalisation. **Recommended for v1.**
- B. *Feed distinctions back into blocker/embedder as training signal.* The right long-term move; adds an ML loop to the engine; defer until the ledger has volume.

5 Failure nodes and the maintenance loop

5.1 The story

The construction thesis of the LossMetricKG work: a graph being built by agents accrues measurable damage, and repair should be targeted at the nodes causing the most harm, the **failure nodes**. Trellis operationalises this with a per-node **health record** combining the four

settled signal families:

Signal family	What feeds it
Construction loss	Extraction disagreement across ingests, property instability, contradiction density. The paper's loss metrics, computed incrementally at write time.
Query-failure attribution	When agent queries fail, return empty, or get corrected downstream, blame propagates to the nodes and edges involved.
Structural anomalies	Supernode detection, orphans, grain mismatches (a blob node with heavy traversal), class-invariant violations.
Staleness / temporal	Time since last confirming ingest, contradictions from newer sources, retired-but-still-referenced status.

Health is queryable data, not a hidden dashboard: `find nodes where health.loss > 0.8 order by query_traffic as worklist`. The maintenance loop is then agent-driven, per the settled principle: an agent (possibly a dedicated maintenance agent) reads the worklist, inspects lineage and evidence, and applies the same verbs everyone else uses: `merge`, `refine`, `retire`, `assert`. Maintenance is not a special mode; it is the same language pointed at the graph's own health data. This is also the cleanest bridge to the "Who Verifies the Agents" paper: the health record is exactly the audit surface a verifier consumes.

5.2 Decisions to make

Decision #11

Health score shape.

- A. *Four named subscores, no combined number.* Agents see `health.loss`, `health.query`, `health.structure`, `health.staleness` and combine as they see fit. Honest, composable, no magic constant. **Recommended.**
- B. *Single combined score with fixed weights.* Simpler worklists, but the weights are arbitrary and workload-dependent.
- C. *Combined score with per-deployment weights.* Configurable magic is still magic; defer until evidence says ranking quality matters.

Decision #12

Query-failure attribution mechanism. The subtle one: how does blame reach a node?

- A. *Explicit agent feedback verb* (`flag` a result or node with a reason). Only counts what agents bother to report; zero inference risk. **Recommended for v1.**
- B. *Automatic: empty results and execution errors increment touched nodes.* Free coverage, noisy blame (an empty result is often a fine answer).
- C. *Both, recorded separately.* The likely end state; start with A, add B behind a separate subscore once eval data shows the noise level.

6 Result serialization and token budgets

6.1 The story

How the engine prints results back to the model matters as much as the query syntax. The evidence is stark: the same knowledge graph costs 2,645 tokens as an edge list, 4,505 as JSON, and 13,503 as JSON-LD, with no accuracy gain for the expensive formats [arXiv 2504.07087]. Encoding choice alone swings graph-task accuracy by tens of points, with per-node “incident” encoding (each node listed once with its edges gathered around it) winning on structural tasks such as degree and neighbourhood questions, though KG-LLM-Bench finds structured formats competitive on accuracy overall, so the format choice must be re-tested in our own eval harness rather than assumed [arXiv 2310.04560, 2504.07087]. Accuracy on cross-referencing collapses as textual distance between related edges grows, up to six-fold [arXiv 2410.01985], and in-context reasoning over large printed graphs degrades as they grow, so results must stay small.

Trellis therefore treats the result format as part of the language spec: incident-style encoding, related edges kept adjacent, relevance-ordered with the most relevant material at the start, declared counts up front as a checkable invariant, and hard caps with explicit truncation markers so the model always knows when it is seeing a window rather than the whole answer. Token budgets are a language-level concept: every query carries a budget (defaulted if unstated), and the engine paginates or summarises rather than overflowing context.

6.2 Decisions to make

Decision #13

Budget overflow behaviour.

- A. *Truncate + continuation token.* Deterministic, cheap, model resumes with `continue <token>`. **Recommended default.**
- B. *Summarise the tail.* An LLM inside the result path adds cost, latency, and a second model’s errors; offer later as an explicit opt-in verb, not a default.
- C. *Refuse over-budget queries.* Punishes the model for not knowing result sizes in advance.

Decision #14

Relevance ordering source. “Most relevant edges first” requires a ranking.

- A. *Query-structural* (distance from the query’s anchor nodes, then recency). Deterministic and explainable. **Recommended.**
- B. *Embedding similarity to the query text.* Better semantic ordering, adds an ANN call to every read.
- C. *None (storage order).* The evidence says this actively harms accuracy; not recommended.

7 Engine architecture

7.1 The story, in plain terms

The settled requirement: the engine is in scope and must scale to billions of datapoints. The honest cost statement: classic distributed databases (own storage, own consensus, own sharding) are multi-year, team-scale builds; Materialize’s SQL frontend alone consumed an estimated 15–20 engineer-years. The modern escape hatch is **disaggregated storage**: keep all durable data in cloud object storage (S3-class: effectively infinite, eleven-nines durable, cheap), keep compute stateless, and put a thin fast log in front for recent writes. Systems built this way recently (Neon for Postgres, SlateDB as an embeddable LSM-on-S3, Turbopuffer for vector search) reach very large scale with very small teams, because the hardest distributed-systems problems (replication, durability, capacity) are bought, not built.

The proposed shape, presented as Approach A in earlier discussion and assumed by the rest of this document:

- **Storage**: an LSM-structured graph layout on object storage. Nodes and their incident edges stored together (matching the incident result encoding, so serialization is nearly a copy). Immutable sorted runs, compacted in the background.
- **Log**: a shared write-ahead log for recent writes and cross-shard ordering.
- **Compute**: stateless query/write executors, one logical writer per shard, local NVMe/RAM caching of hot runs. Autoscaling is just starting more executors; nothing durable lives on them.
- **Background services on the same substrate**: compaction, embedding refresh, ANN index builds, dedup blocking, health-score computation. All read the same immutable runs; none contend with the write path.
- **v0 = the same architecture in one process** (local disk standing in for the object store). No throwaway single-node engine, no later rewrite; scaling out swaps the storage client, not the design.

The language compiles to a logical plan IR (a small algebra of the verb set), the IR is validated against the live schema (the verify-before-execute step that makes self-repair loops work), and then executes against this substrate. The IR is the stable interface: the same IR could also transpile to Cypher or SQL/PGQ for a comparison backend in the eval harness.

Plain words

Object storage / S3: cloud storage for immutable files; slow per request, unlimited in size, extremely durable, very cheap. **LSM**: log-structured merge tree; a write-friendly layout where data lands in sorted immutable files that get merged in the background; the design under RocksDB and most modern write-heavy stores. **WAL / shared log**: an append-only record of every write, the source of truth for “what happened most recently”. **Stateless compute**: servers that hold no durable data, so any of them can die or multiply freely. **Shard**: one partition of the data, small enough for one writer to own. **IR**: intermediate representation; the compiler’s internal, syntax-free form of a query.

7.2 Partitioning: the graph-specific hard part

Graphs resist partitioning because queries walk edges, and a walk that crosses shards pays network cost. Three standard strategies: **hash partitioning** (nodes assigned by ID hash; simple, balanced, every multi-hop crosses shards), **edge-cut / locality partitioning** (keep neighbourhoods together; great for traversals, drifts unbalanced as the graph grows), and **vertex-cut** (split high-degree nodes across shards; the standard answer to supernodes). Two Trellis-specific facts soften the problem. First, token budgets cap every result, so reads are bounded walks, not analytics scans; a caching compute layer covers hot neighbourhoods well. Second, the class system gives a natural partitioning hint: co-partition by class then locality, and let **compact** (agent-driven) shrink supernode neighbourhoods organically.

7.3 Decisions to make

Decision #15

Storage substrate for v1.

- A. *Build the LSM-graph layer over SlateDB (embeddable LSM-on-object-storage, Apache-2.0).* Inherits durability and compaction machinery; graph layout becomes a key-encoding design. **Recommended starting point, prototype required.**
- B. *Raw object storage + custom LSM.* Full control, most work; fall-back if SlateDB's abstractions fight the graph layout.
- C. *FoundationDB.* Proven transactional substrate, but ordered-KV transactions at graph scale bring their own tuning burden and it is not object-store-native.

Decision #16

Consistency model.

- A. *Per-shard strong, cross-shard eventual, causal tokens in receipts.* Each write receipt carries a position; an agent's next query can say **after** <position> to read its own writes. Matches agent interaction patterns (sequential tool calls). **Recommended.**
- B. *Global strong consistency.* Simplest semantics, pays cross-shard coordination on the hot path at billions scale.
- C. *Fully eventual.* Agents observably lose their own writes; poisonous for construction loops.

Decision #17

Partitioning for v1.

- A. *Hash by node ID, rely on caching + budget-bounded reads.* Simple, balanced, measurable; add locality later if evals show cross-shard walks dominate. **Recommended.**
- B. *Class + locality co-partitioning from day one.* Better traversal locality, real rebalancing complexity before evidence demands it.

Decision #18

Vector index for dedup embeddings.

A. *Per-class IVF/HNSW indexes built as background jobs over the LSM runs, stored as object-storage artifacts.* Same pattern as Turbopuffer; consistent with the substrate.

Recommended.

B. *External vector DB.* Faster to first demo, second system to operate, second consistency domain.

8 Evaluation plan

The eval harness is both the steering instrument and the paper. The design, CypherBench-style: same questions, same graphs, Trellis versus text-to-Cypher, execution accuracy as the metric. Slices that matter: multi-hop and compositional queries (where Anka predicts a win), one-to-two-hop lookups (where the honest expectation is parity at best), and, novel to this project, **construction benchmarks**: give agents a document stream, measure resulting graph quality (duplicate rate, contradiction rate, loss-metric trajectory) under Trellis structural writes versus an episodic-ingest baseline (Graphiti-style). Held-out schemas guard against future contamination once models train on published Trellis examples.

Sequencing, and the cheapest possible falsification first: before any compiler or engine code, run the language *purely in context*. Grammar prompting on a CypherBench slice: Trellis sketch + grammar + examples in the prompt, hand-translate the gold queries, execute via a mechanical Trellis-to-Cypher mapping. If in-context Trellis does not beat text-to-Cypher on multi-hop slices, the language design needs iteration before anything else is built. This was already the agreed next step for the project, and every decision box in this document is testable at that stage except the engine ones (15–18), which activate only after the language clears the bar.

9 Consolidated decision checklist

#	Area	Question	Recommended
1	Language	Verb count / surface tiering	B: two-tier
2	Language	Edge direction encoding	A: role-named
3	Language	Cold-start strategy	A+B
4	Language	Pattern-match semantics	A: fixed default
5	Granularity	State model	A: blob/composite/atom
6	Granularity	Refinement mapping source	A: agent-supplied
7	Granularity	Lineage retention	A: full, cold storage
8	Writes	Merge reconciliation	A: per-merge policy
9	Dedup	Detection latency	A: sync blocking, async ANN
10	Dedup	Distinct assertions	A: pair-level
11	Maintenance	Health score shape	A: four subscores
12	Maintenance	Failure attribution	A: explicit flag verb
13	Results	Budget overflow	A: truncate + continue
14	Results	Relevance ordering	A: query-structural
15	Engine	Storage substrate	A: SlateDB base
16	Engine	Consistency	A: per-shard strong + causal tokens
17	Engine	Partitioning	A: hash + caching
18	Engine	Vector index	A: native background-built

Decisions 1–14 gate the in-context prototype and the language spec. Decisions 15–18 gate the engine and can safely wait for prototype results, though substrate spikes (15) can start in parallel.

10 Annotated reading list

Ordered by how much each will change your decisions, most first.

1. **CypherBench** [arXiv 2412.18702]. The benchmark and the error taxonomy behind decisions 2–4. Read the error analysis section closely; it is the target list.
2. **Anka** [arXiv 2512.23214]. The existence proof that a designed DSL beats a familiar language, and the source of the named-intermediates and canonical-form rules. Also the source of the honest caveat about 1–2 hop tasks.
3. **Talk like a Graph** [arXiv 2310.04560] and **KG-LLM-Bench** [arXiv 2504.07087]. Together they frame the result-format questions (decisions 13–14); note they disagree on the best format (incident-style vs structured JSON, task-dependent), which is why the format is re-tested in our eval harness.
4. **Lost-in-Distance** [arXiv 2410.01985]. Why edge adjacency in output matters; short read.
5. **GraphRunner** [arXiv 2507.08945]. Plan-verify-execute beating per-hop agent loops by 10–50% at 3–13x lower cost; the argument for verify-before-execute as a core language feature.
6. **Grammar Prompting** [arXiv 2305.19234] and **SPEAC** [arXiv 2406.03636]. The two escape routes from zero corpus (decision 3).
7. **XGrammar** [arXiv 2411.15100] plus the dottxt rebuttal (blog.dottxt.ai/say-what-you-mean.html). Constrained decoding is cheap and does not hurt reasoning; grounds the schema-closed grammar plan.
8. **Zep/Graphiti** [arXiv 2501.13956] and **Mem0** [arXiv 2504.19413]. What the episodic-write competition does, and therefore what the construction benchmark baseline is.
9. **Against SQL** (scattered-thoughts.net/writing/against-sql). The effort economics of language frontends; the argument for a small denotationally simple surface. Directly motivates decision 1.
10. **SlateDB** (slatedb.io) and **Neon architecture** (neon.tech/docs/introduction/architecture-overview). The disaggregated-storage pattern for decision 15; read SlateDB’s design doc before the substrate spike.
11. **DuckPGQ** [arXiv 2505.07595] and **Formal Semantics of Cypher** [arXiv 1802.09984]. The transpile-comparison backend for the eval harness, and the template for writing Trellis’s formal semantics (decision 4).
12. **Text2GQL-Bench** [arXiv 2602.11745]. Evidence that unfamiliarity, not capacity, is the barrier, and that small fine-tunes close the gap (decision 3D).
13. **TOON generation study** [arXiv 2603.03306]. The prompt-tax threshold result; keeps the tutorial honest about when Trellis’s syntax actually pays.
14. **GraphRAG-Bench** [arXiv 2506.05690]. The skeptic’s case that graphs often lose to vanilla RAG; defines the use cases Trellis must win to matter.